

Name of the file	Type/Content
1. <u>Rule Based Strategy_DRS</u>	<u>Folder</u>
1.1 Controller_DRS_SS_23	<i>Matlab code</i> / This function is responsible for calculating the torque split factor based on various conditions and inputs such as flywheel angular velocity, acceleration, torque, and battery charge.
1.2 qss_hybrid_electric_vehicle_DRS	<i>Simulink model</i> / model based on QSS toolbox to determine equivalent fuel consumption and state of charge for both NEDC and FTP-75.
1.3 start_stop_DRS_SS_23	<i>Matlab code</i> / Determines the state of the Combustion Engine based on the flywheel torque and the torque-split factor for both NEDC and FTP-75.
2. <u>Dynamic programming_DRS</u>	<u>Folder</u>
<u>2.1 NEDC</u>	<u>Sub-Folder</u>
2.1.1 DP_NEDC_main	<i>Matlab code</i> / This function is primarily responsible for setting up and executing dynamic programming (DP) optimization for a driving cycle known as NEDC (New European Driving Cycle).
2.1.2 DP_NEDC	<i>Matlab code</i> / This function is designed to handle the initialization, simulation, and propagation of parameters related to the NEDC.
2.1.3 dpm	<i>Matlab code</i> / Function to perform dynamic programming-based optimization.
2.1.4 BT	<i>Matlab data</i> / consists of battery parameters
2.1.5 State_CE_DP_DRS	<i>Matlab code</i> / This function determines the state of the engine (either on or off) based on the given input parameters related to the torque of the Main Gear Box (MGB) and the split ratio.
2.1.6 NEDC_inputs	<i>Matlab data</i> / consists of paramters such as T_MGB, dw_MGB, v, gear number, w_MGB and x_tot for NEDC.
2.1.7 NEDC_outputs	<i>Matlab data</i> / consists of optimization results and dynamics for NEDC.
2.1.8 torque_split	<i>Simulink model</i> / performs torque split calculations (for both forward and backward propagation) for dynamic programming
2.1.9 qss_hybrid_electric_vehicle_DPM_NEDC	<i>Simulink model</i> / model based on QSS tool box to calculate equivalent fuel consumption and State of Charge for dynamic programming approach for NEDC.
2.2.10 output_visualization_NEDC (A part of file from <i>DP_NEDC_main</i> to have quick visualization of results of Dynamic Programming approach)	<i>Matlab code</i> / directly access the optimization results of NEDC to produce split ratio using optimized torque values. This data can be further interpolated using lookup table to produce results of equivalent fuel consumption.

2.2 FTP-75	Sub-Folder
2.2.1 DP_FTP-75_main	<i>Matlab code</i> / This function is primarily responsible for setting up and executing dynamic programming (DP) optimization for a driving cycle known as FTP-75 driving cycle.
2.2.2 DP_FTP-75	<i>Matlab code</i> / This function is designed to handle the initialization, simulation, and propagation of parameters related to the FTP-75.
2.2.3 dpm	<i>Matlab code</i> / Function to perform dynamic programming-based optimization.
2.2.4 BT	<i>Matlab data</i> / consists of battery parameters
2.2.5 State_CE_DP_DRS	<i>Matlab code</i> / This function determines the state of the engine (either on or off) based on the given input parameters related to the torque of the Main Gear Box (MGB) and the split ratio.
2.2.6 FTP-75_inputs	<i>Matlab data</i> / consists of paramters such as T_MGB, dw_MGB, v, gear number, w_MGB and x_tot for FTP-75.
2.2.7 FTP-75_outputs	<i>Matlab data</i> / consists of optimization results and dynamics for FTP-75.
2.2.8 torque_splits	<i>Simulink model</i> / performs torque split calculations (for both forward and backward propagation) for dynamic programming.
2.2.9 qss_hybrid_electric_vehicle_DPM_FTP_75	<i>Simulink model</i> / model based on QSS tool box to calculate equivalent fuel consumption and State of Charge for dynamic programming approach for FTP-75 driving cycle.
2.2.10 output_visualization_FTP_75 (A part of matlab code file from DP_FTP-75_main to have quick visualization of results of Dynamic Programming approach)	<i>Matlab code</i> / directly access the optimization results of FTP-75 to produce split ratio using optimized torque values. This data can be further interpolated using lookup table to produce results of equivalent fuel consumption.

General Note 1: QSS toolbox folder is included in both the approaches individually to prevent the override and crashes during simulation of code. And in dynamic programming, I have made the files individually under NEDC and FTP-75. While checking out the results of Dynamic Programming approach, kindly check the results of NEDC and FTP-75 separately as the files and folders are organized in individual fashion.

General Note 2:

- The optimization results of dynamic programming are already saved under NEDC_outputs and FTP-75 outputs respectively.
- The last section of code from DP_NEDC_main and DP_FTP-75_main are separately stored under the files named *output_visualization_NEDC* and *output_visualization_FTP_75* respectively.
- In order to check the simulation results for equivalent fuel consumption directly, you can run the file named *output_visualization_NEDC* or *ouput visualization_FTP_75* and then run the

model in QSS toolbox. Or, you can also run the last section of code from *DP_NEDC_main* (or) *DP_FTP-75_main* and proceed with simulation of model in QSS toolbox.

- If you wish to check the whole optimization process, you can run the whole script named *DP_NEDC_main* or *DP_FTP-75_main*.

FUNCTIONALITY OF THE CODE

A. RULE BASED STRATEGY:

Controller is designed in such a way that two matlab interpreted functions are used which plays a role in estimating the equivalent fuel consumption and battery state of charge especially for NEDC and FTP-75 driving cycles. Simulation is done using QSS toolbox. Functionalities of two matlab interpreted functions are shown below.

1. Controller DRS SS 23:

Inputs:

- *w_MGB*: Flywheel angular velocity
- *dw_MGB*: Flywheel angular acceleration
- *T_MGB*: Flywheel torque
- *Q_BT*: Current battery charge

Operations:

- The function takes into account the maximum motor angular velocity (*w_EM_max*), maximum motor torque (*T_EM_max*), and initial battery charge (*Q_BT_IC*).
- Depending on the stop-time of the driving cycle (*stop_time*), specific parameters for the NEDC or FTP-75 are set.
- Based on the values of *T_MGB* and *Q_BT*, the function determines which control mode the system should be in:
 - Regeneration Mode
 - Load Point Shifting in Motor Mode
 - Electric Drive Mode
 - Load Point Shifting in Generator Mode
 - Engine Mode
- The output *u* represents the torque-split factor, which is calculated based on the identified control mode.

2. start_stop DRS SS 23:

Inputs:

- *T_MGB*: Flywheel Torque
- *u*: Torque-split factor

Operations:

- This function determines the state of the Combustion Engine (state_CE).
- If the Flywheel Torque (T_MGB) is less than 0 or the Torque-split factor (u) equals 1, the state of the Combustion Engine is 0 (OFF).
- Otherwise, the state of the Combustion Engine is 1 (ON).

B. DYNAMIC PROGRAMMING APPROACH:

- The function **DP_NEDC** is primarily designed to simulate a hybrid vehicle's engine and motor operation under specific driving demands using a Simulink model, and then determine the state-of-charge of the battery, fuel consumption, and system feasibility based on the simulation results.
- The function **DP_NEDC_main** optimizes a driving cycle using dynamic programming.
- Finally, the simulation takes place in QSS toolbox by making use of optimized values from dynamic programming to estimate the value of equivalent fuel consumption and battery state of charge.

General Note for Dynamic Programming: I have explained the functionalities of both the functions separately and it is explained in terms of *NEDC* driving cycle. The code functionality remains the same also for *FTP-75* driving cycle.

1) DP_NEDC function:

- (i) Function Signature: function [X, C, I, out] = DP_NEDC (inp, ~)
 - The function takes two inputs: inp (input structure) and ~ (a placeholder, suggesting that the second parameter might not be used in this specific version of the function).
 - It returns four outputs: X (resulting state-of-charge), C (cost matrix), I (infeasible matrix), and out (user-defined output signals).
- (ii) Initialization:
 - Speed, acceleration, and torque values are extracted from the inp structure.
 - Engine parameters are loaded from the file OM_622.mat, and certain variables are initialized.
 - Electric motor parameters are loaded from the file EM, and certain variables are initialized.
 - Battery parameters are loaded from the file BT, and other related variables are initialized.
 - Some dynamic parameters like engine start-stop status (u1) and split ratio (u2) are initialized.
- (iii) Loading Simulink blocks:
 - The Simulink model named torque_split is loaded, which presumably simulates the torque distribution between the engine and the electric motor.
 - A ConfigSet object named configObj is created to configure the Simulink model simulation settings.
- (iv) Propagation:

Depending on the size of u1, different logic is applied:

 - If u1 is not a single row, the code reshapes certain matrices and runs the Simulink model. Once the simulation is complete, it reshapes the output variables back to 2D.

- If u_1 is a single row, the code directly runs the Simulink model without reshaping the inputs and outputs.
 - In both cases, after running the Simulink model, infeasibility calculations are performed based on the simulation results.
- (v) Infeasible Matrix Calculation:
- I is computed by combining infeasibility from the controller, engine, motor, and battery.
 - It's essentially a logical matrix indicating where the system might not operate correctly.
- (vi) Update State Variable:
- The state-of-charge ($X_{\{1\}}$) of the battery is updated based on the simulation results.
- (vii) Cost Matrix:
- $C_{\{1\}}$ is calculated as the fuel consumption (Pe_{out}), which likely denotes how much fuel the engine consumed during the operation.
- (viii) Infeasibility Check:
- If the system is found to be completely infeasible (i.e., all entries in I are 0), a keyboard command is executed. This would halt the function and allow for debugging within MATLAB's command window.
- (ix) User-Defined Output Signals:
- The torques of the engine (Te_{out}) and the motor (Tm_{out}) are assigned to the 'out' structure.

2) DP_NEDC_main function:

- (i) Initialization:
- Start a stopwatch timer to measure the execution time.
 - Load the NEDC driving cycle inputs.
 - Set up the grid for state of charge (SOC) and split ratio boundaries and discretization.
 - Specify the initial and final SOC values.
 - Define optimization parameters, including speed, torque, and acceleration data.
 - Configure options for dynamic programming method, specifying boundary method and tolerance.
- (ii) Optimization:
- Call the dynamic programming function to perform optimization for the NEDC driving cycle.
 - Measure the execution time taken for optimization.
- (iii) Post-Processing:
- Save the optimization results and dynamics in an output MAT-file.
 - Calculate and save torque split ratio for the Simulink model.
 - Store the calculated split ratio and time in a MAT-file and Excel sheet for further processing.